PLAIN-LANGUAGE IMPLEMENTATION LOG

MedVault Implementation Summary

Record date: 11 October 2026 | Asia/Manila
The complete explanation list, verification results, and remaining deployment work.

The application changes are implemented. Ubuntu server protections and the daily backup schedule are prepared and still need activation on the deployment host.

Patient information and current setup

01 **Your current roles stay the same.**

Super Admin, Receptionist, and Medical Secretary keep their existing permissions.

02 **SQLite stays as your database.**

PostgreSQL setup is postponed.

03 **Medical Info shows saved information.**

When the Medical Secretary saves medical history or consultation details, the Receptionist can see them. Fields without saved information stay blank.

04 **Patients have New/Past indicators.**

New Patient means there are no saved medical details or previous visits yet. Past Patient means medical information or visit history already exists. Booking an appointment alone does not make someone a past patient.

05 **Passwords are protected.**

The system stores a protected password hash rather than the actual password. Existing staff passwords continue to work.

IMPLEMENTATION EXPLANATION / CONTINUED

Login and record security

These protections run in the application. Production session security also depends on the HTTPS deployment.

06

Repeated incorrect passwords temporarily block sign-in.

After five failed attempts, that username is locked for 15 minutes.

07

Inactive users are automatically signed out.

Sessions expire after 30 minutes without activity, or eight hours total. Automatic dashboard refreshes do not keep an inactive user signed in.

08

Login sessions are harder to misuse.

The system checks that the login session is valid. Signing out, changing a password, deactivating an account, or changing its role cancels affected sessions.

09

Permissions are checked by the server.

Someone cannot gain extra access simply by changing the webpage or sending a request directly. For example, a Receptionist cannot edit clinical information.

10

Saved medical information is encrypted.

Sensitive information is stored in a scrambled form that requires the correct encryption key to read.

11

Forms and save requests have extra protection.

The system removes harmful script markup, prevents inputs from becoming database commands, and blocks another website from secretly submitting changes through your login. Normal medical text, including characters such as <, > and &, is preserved.

IMPLEMENTATION EXPLANATION / CONTINUED

History, backups and server setup

Record changes are traceable. Recovery tools are included, with server activation still to be completed.

12

Changes leave an audit history.

The system records who signed in or changed information, what action happened, and when. Audit entries have checks that help detect alteration, and ordinary updates or deletions of the audit log are blocked.

13

Encrypted backup and restore tools are included.

Backups protect the saved database. Restoring checks the backup before replacing existing data and preserves a copy of the previous database. A daily backup schedule is prepared.

14

Secure server deployment files are prepared.

These configure encrypted HTTPS connections, firewall restrictions, safer SSH access, traffic limits, and one-hour IP bans for repeated failed logins. They also limit container resources and configure automatic restarts after process failures.

15

Testing and documentation are included.

All 14 backend tests, frontend tests, build, and lint passed. Dependency checks found no known vulnerabilities. The documentation explains permissions, risks, backup steps, and deployment checks.

Status reminder: the Windows application work and local checks are complete. Live HTTPS, firewall, SSH, Fail2ban, Docker and daily cron checks belong to the Ubuntu deployment.

Keep encryption keys safe and separate from backups. The matching key is needed to read encrypted medical records or restore an encrypted backup.

VERIFICATION AND REMAINING WORK

Verification record

These results were recorded during the implementation work on 11 October 2026.

Check	Recorded result
Backend security and workflow tests	**14 tests passed**
Frontend data synchronization tests	**2 tests passed**
Production frontend build	**Passed**
Code lint and syntax checks	**Passed**
Sign-in through the local Vite server	**Passed**
Existing SQLite records and staff upgrade	**Passed without data loss**
Encrypted backup and restore checks	**Passed**
Audit immutability and tamper checks	**Passed**
Server and client dependency audits	**0 known vulnerabilities**

Local performance test

200/200	20	280.64 ms
successful requests	concurrent clients	95% completed within

Median response: 177.38 ms. 99% completed within 288.29 ms. Throughput: 106.26 requests/second. This was a local synthetic SQLite test on Node.js v24.16.0; live server performance still needs testing.

Still required on the Ubuntu deployment host

- Activate HTTPS certificates, firewall rules, key-based SSH and Fail2ban; verify public ports and traffic limits.
- Enable the daily encrypted backup job, off-host backup copies and protected key storage; perform a recovery drill.
- Check container restarts and run a staging security scan and representative performance test.

Project records: docs/SECURITY.md; deploy/README.md; output/security/load-results.json. PostgreSQL setup remains deferred.